

CookMate - AI Agent

Zuzanna Makowska

zuza.m.makowskal@gmail.com

Abstract

This project presents CookMate, a simple cooking advice agent built by extending an initial agent template based on the [Agents template by Birger Moell](#). The project repository is available on GitHub: [CookMate repository](#).

The goal of the project was to improve the agent's ability to use context through simple information retrieval methods. To achieve this, CookMate was extended with persistent memory tools that store and retrieve user cooking preferences, such as diet, allergies, liked cuisines, time constraints, or other details. Before giving recipe suggestions, the agent can retrieve these preferences and adapt its response to the user's needs. The system also allows users to explicitly save recipes as favorites and later search through them. This shows how a lightweight agent can become more personalized by combining language model interaction with memory and retrieval tools.

1 Implementation

1.1 Rewriting the template agent in Python

The implementation was divided into four main steps. First, the initial agent template was adapted into a Python-based version. The original template can be found in [Agents template repository](#). This step was supported with Codex, which helped translate the main agent structure into Python and adapt it to an OpenAI-compatible chat completion API. The resulting template included a basic command-line interaction loop, system prompt handling, and support for tool calling.

1.2 Memory

The second step was to add persistent memory. For this purpose, a new memory.py file was created. This file handles reading and writing JSON files stored in the data/ directory. The memory system stores two types of information: user cooking preferences and favorite recipes. User preferences

include information such as diet, allergies, liked cuisines, disliked ingredients, preferred cooking time, and other notes such as for example available or unavailable equipment. Favorite recipes are stored only when the user explicitly asks the agent to save a recipe.

1.3 Extending the tools

The third step was to extend the available tools. New memory-related tools were added to tools.py, including tools for retrieving and updating user preferences, saving favorite recipes, searching favorite recipes, and removing recipes from favorites. These tools allow the language model to interact with the persistent memory system. In this way, memory lookup and favorite recipe search act as simple information retrieval components.

1.4 System prompt

The final step was to update the system prompt. The prompt was changed to a cooking advice agent prompt. The new prompt instructs the agent to retrieve stored user preferences before giving recipe suggestions, update preferences when the user provides stable cooking-related information, and save favorite recipes only when explicitly requested. This makes the agent's behavior more consistent with the goal of using retrieved context to provide personalized cooking advice.

2 Example Interaction

An example interaction demonstrates how CookMate uses memory and retrieval during a conversation. First, the user provides stable cooking preferences, such as being vegetarian, allergic to lactose, liking Italian and Mexican cuisine, not having an oven, and preferring quick recipes. The agent stores this information in persistent memory and can reuse it in later turns. When the user later asks for dinner advice, CookMate retrieves the stored preferences before answering and suggests recipes

that match these constraints. If the user likes one of the suggested recipes, they can explicitly ask the agent to save it as a favorite. The recipe is then stored in the favorites memory and can later be retrieved through keyword search, for example by asking for favorite recipes containing a specific ingredient. A demo of this interaction can be found in the project [README](#).

3 AI Assisted Development and Reflection

AI tools were used at different stages of the project, including initial code adaptation, implementation, code understanding, and documentation. Since the starting point for the project was an existing agent template, one of the first steps was adapting the initial repository into a Python-based version. Codex was especially helpful at this stage, as it significantly sped up the process of translating the structure of the original template into Python and adjusting it to the needs of this project.

ChatGPT was also used during the early stages of the work to better understand the codebase written by another author. It helped provide a quicker overview of the project structure, files, and the way different parts of the agent were connected. This made it easier to decide where the memory functionality should be added and how the new tools could fit into the existing implementation.

In the later implementation phase, AI tools were mainly used to support documentation and small repetitive tasks, especially related to JSON handling in the memory tools. This helped speed up the process, but the final changes still required manual testing to verify that preferences and favorite recipes were correctly saved and retrieved.

Finally, ChatGPT was used to support writing the report and refining the README file. The generated content was helpful as a starting point, but it required (quite a lot of ;) manual adjustments to better match the writing style natural. Overall, AI tools made the development process faster and helped with both coding and explanation, but the outputs still had to be carefully reviewed to ensure that they matched the project assumptions and did not go beyond the intended scope of the task.